PROGRAMMING AI ACCELERATORS · EVALUATION 1

Where Do the Joules Go in Speculative Decoding?

Speculative decoding is tuned for latency. We measure its cost in joules per token, component by component, and show the draft length that runs fastest is not the one that runs leanest.

Anupam Dwivedi · Ankit Kumar · Kanishk Kumar

IIIT Hyderabad · October 2026

7,488 measured generations, all lossless-gated
2 target sizes · 2 precisions · 2 GPU generations

Problem and goal

Speculative decoding lets a small **draft** model propose K tokens and a large **target** model verify all K in one forward pass. Latency falls 2–3×. Energy does not follow, because every **rejected** draft token was computed and then thrown away.

METRIC WE IMPROVE

mJ / token

Energy per generated token at **fixed output**. Greedy speculative decoding is lossless, so quality is held exactly constant and energy is the only free variable.

THE KNOB

draft length K

Larger K amortises the target forward pass over more tokens, but acceptance falls, so more drafted tokens are wasted. Two opposing forces, so an optimum exists.

THE CLAIM TO TEST

K*energy ≠ K*speed

If the two optima differ, every deployment tuned for throughput is silently overspending on energy, and by a measurable amount.

Qwen2.5-3B ← Qwen2.5-0.5B · GSM8K · greedy, batch 1 · protocol fixed to Dutta et al., Findings of EACL 2026

Qwen2.5-7B: the same two rounds at K=4 and K=8

K = 4 · 44% OF DRAFTED TOKENS THROWN AWAY

round 0 K = 4 accepted 1/4 emitted 2 draft 287.8 ms verify 21.1 ms

To ·determine ·the ·average ·find

round 1 K = 4 accepted 3/4 emitted 4 draft 22.6 ms verify 51.4 ms

·the ·average ·of ·the ·Jared

K = 8 · 62% THROWN AWAY

round 0 K = 8 accepted 1/8 emitted 2 draft 323.8 ms verify 21.3 ms

To ·determine ·the ·average ·of ·the ·three ·measurements ·find

round 1 K = 8 accepted 3/8 emitted 4 draft 48.0 ms verify 50.4 ms

·the ·average ·of ·the ·three ·measurements ·, ·we ·Jared

Qwen2.5-7B ← Qwen2.5-0.5B, fp16, greedy · captured from the real engine with token logging on · first 64 generated tokens, output verified against plain greedy decoding

PROMPT · GSM8K TEST #588

Jared is trying to increase his typing speed. He starts with 47 words per minute (WPM). After some lessons the next time he tests his typing speed it has increased to 52 WPM. If he continues to increase his typing speed once more by 5 words, what will be the average of the three measurements?

- **accepted**
- **rejected** here
- **discarded**, never judged
- **bonus**, from the target

Round 0 accepts **one** token either way. Doubling K bought nothing there and cost four extra draft forward passes — all grey.

Rejection is a **prefix** rule: once the target disagrees, everything drafted after it is unusable, however good it was.

Both runs emit **byte-identical text**. K changes the energy and the timing and nothing else — losslessness, on a concrete example.

Qwen2.5-3B: a smaller target wastes just as much

K = 4 · 44% OF DRAFTED TOKENS THROWN AWAY

round 0 K = 4 accepted 1/4 emitted 2 draft 275.1 ms verify 13.4 ms

To ~~·determine~~ ~~·the~~ ~~·average~~ ·find

round 1 K = 4 accepted 3/4 emitted 4 draft 22.7 ms verify 43.8 ms

·the ·average ·of ~~·the~~ ·Jared

K = 8 · 66% THROWN AWAY

round 0 K = 8 accepted 1/8 emitted 2 draft 295.4 ms verify 13.5 ms

To ~~·determine~~ ~~·the~~ ~~·average~~ ~~·of~~ ~~·the~~ ~~·three~~ ~~·measurements~~ ·find

round 1 K = 8 accepted 3/8 emitted 4 draft 45.3 ms verify 44.3 ms

·the ·average ·of ~~·the~~ ~~·three~~ ~~·measurements~~ ~~,~~ ~~·we~~ ·Jared

wasted, same prompt	K = 4	K = 8
7B target	44%	62%
3B target	44%	66%

Essentially the same waste from a target less than half the size. **How much** is discarded is set by the draft model and by K — not by how big the target is.

> What target size changes is the **price** of that waste. A discarded token costs one draft pass either way, but the verify pass it was thrown against is far more expensive at 7B — which is why the optima sit at different K.

All four traces use **GSM8K test #588**, the same seed and the same 0.5B draft, so they compare chip for chip. The two targets answer in different words, as two different models should; within each target, K=4 and K=8 are byte-identical.

Qwen2.5-3B ← Qwen2.5-0.5B, fp16, greedy, RTX PRO 5000 · waste = drafted tokens not accepted, over the whole 64-token generation

What is new

	Dutta et al., EACL 2026	This project
Question	Does speculative decoding save energy?	Where inside it do the joules go, and which K is leanest?
Instrument	CodeCarbon, sampled at 1 Hz	NVML energy counter read around each phase, plus CLEAR amplification
Granularity	Whole model, whole run	Draft vs verify, then attention / MLP / lm_head / cache rollback
Blind spot	A draft phase and a verify phase are milliseconds apart — 1 Hz cannot separate them	Separates them, and charges the wasted work of rejected tokens to a component

THE GENUINELY NOVEL STEP

CLEAR has only ever been applied to **single-model** inference, where every joule produces output. A draft–verify pipeline has a second model and a category the method has no notion of: computation that produced no token. Extending component attribution across that boundary is the contribution.

Why it is worth knowing

A phase-level answer says *which K*. A component-level answer says *why* — and therefore which K to pick on a card nobody has benchmarked yet.

Leviathan et al., ICML 2023 · Chen et al., arXiv 2023 · Jain et al., CLEAR 2025 · Dutta et al., arXiv:2602.09113

The baseline, and the gate it had to pass

WHAT WE COMPARE AGAINST

Plain autoregressive greedy decoding on the target model — the system speculative decoding has to beat, measured on the same prompts, same card, same session.

Dutta's protocol, held fixed: Qwen2.5 pairs, NF4 and fp16 targets, greedy, batch 1, 256 GSM8K prompts, 256 new tokens, seed 42. Our numbers sit beside theirs rather than in place of them.

4,608	2 / 2	3.6%
generations measured	sweeps complete	meter rel. std dev

THE CORRECTNESS GATE

Greedy speculative decoding is **lossless**: it must emit exactly the tokens plain greedy would. One wrong logit row or one stale KV entry breaks that silently, and the energy numbers would then describe a different decoder.

We assert token-identical output across every strategy and four draft qualities — including a draft identical to the target (must accept 100%) and an unrelated one (accepts 0%, exercising the reject path every round).

> On GPU the equality holds only to floating-point ordering. A diagnostic compares the top-2 logit gap against the numerical spread, so a tie is never mistaken for a defect.

COGA-1…8 plus autoregressive, at fp16 and NF4, on a Tesla T4 · both runs 2,304 / 2,304 units complete

Baseline result: the two optima are two tokens apart

K	Accept	Draft share	tok/s	mJ/token
1	92.3%	39.8%	18.75	1864
2	89.0%	57.4%	19.62	1590
3	85.3%	66.8%	19.99	1175
4	82.0%	72.9%	19.87	927
5	79.4%	77.1%	19.69	861
6	76.3%	80.1%	19.23	869
7	73.7%	82.5%	18.81	889
8	71.1%	84.3%	18.34	915

LATENCY OPTIMUM

K = 3

ENERGY OPTIMUM

K = 5

Running the latency-optimal K costs **+320 ± 14 mJ/token — 36.6% more energy** for 1.5% more throughput. Paired across the same 256 prompts, so difficulty cancels; the two sets do not overlap.

Qwen2.5-3B ← 0.5B, fp16, GSM8K, Tesla T4 · mJ/token above a measured 27.3 W idle floor · best raw energy 0.56× autoregressive

One variable at a time: the hardware decides the ordering

Target ← draft, device	n	plain greedy	speed-up	fastest K	leanest K	penalty
3B ← 0.5B · Tesla T4 70 W cap	256	16.1 tok/s	1.24×	3	5	+36.6%
3B ← 0.5B · RTX PRO 5000 300 W cap	160	79.8 tok/s	1.30×	5	3	+14.0%
7B ← 0.5B · RTX PRO 5000 300 W cap	160	50.1 tok/s	1.88×	7	6	+6.9%

Swap the card, hold the model

Rows 1→2. The ordering **inverts**: the leanest K goes from *above* the fastest to *below* it. The sign of the gap is a property of the hardware.

Swap the model, hold the card

Rows 2→3. Both optima slide up together, 5→7 and 3→6, and the penalty halves. Size moves the curve; it does not reorder it.

WHY: THE POWER CAP

On the T4, average power **falls 68 → 44 W** as K grows — plain decoding pins the card at its 70 W cap and drafting relieves it, so energy keeps improving past the throughput peak. On the 300 W card power is **flat at 111–138 W**, so energy tracks 1/throughput and the two optima nearly coincide.

All three runs complete, fp16, GSM8K, greedy, batch 1, seeded prompt prefix · penalty = paired per-prompt energy at the fastest K vs the leanest, 95% CI

Two findings the baseline already forces on us

1 · NF4 BUYS MEMORY, NOT ENERGY

At best K	tok/s	mJ/tok	VRAM
fp16	19.99	2181	6.82 GB
NF4	15.96	3080	2.93 GB

fp16 is **29% leaner and 25% faster**: a T4 has no 4-bit arithmetic, so every weight is dequantized before the matmul. Any energy claim about quantized speculative decoding must name the card.

> NF4's curve has not turned by K=8, so its optimum lies outside what we swept. The analysis flags the boundary instead of reporting an edge as a minimum.

2 · THE METRIC CHOICE CHANGES THE VERDICT

Raw counter
"my bill"
0.80×

Above idle
"how much work"
1.04–1.32×

At NF4 the same data says speculative decoding **saves 20%** or **costs more at every K**, depending on whether idle draw is subtracted. Both are correct.

NF4 plain decoding is memory-bound and sits at **44.5 W** against a 32.7 W floor; speculative pushes the card to **51 W** but finishes sooner. The saving is a shorter clock, not less computation.

> At fp16 both metrics agree on K=5. They diverge exactly where the work barely lifts the card off idle — which is where component attribution is needed to settle it.

Ratios against autoregressive decoding on the same prompts · idle floor measured per process before each sweep

Proposed approach: amplify what is too small to measure

One attention block runs in microseconds. The GPU's energy counter updates every ~100 ms. CLEAR closes that four-order gap by re-executing a component thousands of times on cached inputs and dividing.

STEP 1

Capture

Hooks on both models record each component's input activations during a real draft–verify round, tagged with the round and the token position.

STEP 2

Amplify

Replay that one component N = 10^3–10^4 times on its cached input, until the run clears ~1 s. Per-call energy is (E – Pidlet) / N.

STEP 3

Attribute waste

Each replay carries the accept/reject outcome of the token it served, so component energy splits into **useful** and **discarded** joules.

STEP 4

Check it adds up

%Capture = Σ component energy ÷ measured phase energy. Below 90% the decomposition is incomplete and we say so rather than report it.

Why it should work here

The measurement precondition is already met, not assumed: our probe measures **1.6–3.6% relative std dev** at 1 s windows across the cards we have tested, comfortably inside CLEAR's 10% bar, with the counter cadence measured at ~100 ms. The activation store reuses the same engine the baseline runs on, so the components replayed are literally the ones that were measured.

What it buys

A phase number says *which K* on one card. A component model says *why* — and lets us predict K* on a card we have not benchmarked, from its component costs and an acceptance curve.

Components: embedding, QKV projection, attention core, MLP, layernorm, lm_head · plus KV-cache rollback and the verification comparison

Experimental plan

Question	Comparison	State
Is the energy-optimal K the throughput-optimal K?	COGA-1…8 vs autoregressive, two target sizes	done · no, in both
Does quantization change the answer?	fp16 vs NF4, identical sweep	done · yes, on Turing
Where is NF4's optimum, past the swept edge?	extend to COGA-12, same card and prompts	planned
Does K* move with target size?	1.5B / 3B / 7B ← 0.5B, draft fixed, fp16	3B and 7B done
Is K* a property of the hardware?	Turing vs Blackwell, 3B fp16, identical sweep	done · yes, it sets the ordering
Which component dominates, and how much is waste?	CLEAR attribution at K ∈ {1, 3, 5, 8}	next phase
Ablation: does adaptive K beat the best fixed K?	DYGA-{1,4,20} as scatter points against the COGA curve	implemented
Ablation: does the accounting metric change K*?	raw counter vs above-idle, every run	instrumented

TRADE-OFF 1
Latency against energy — 1.5% throughput for 36.6% energy, at 3B fp16

TRADE-OFF 2
Memory against energy — NF4 is 2.3× smaller and 29% thirstier

TRADE-OFF 3
Draft cost against acceptance — the force that sets where K* lands

Every row uses the same 256 GSM8K prompts, seed 42 · smaller runs take a prefix of that set, so all comparisons stay paired

How we will know whether it worked

SUCCESS CRITERIA, FIXED IN ADVANCE

Criterion	Bar	Now
Losslessness gate	token-identical	pass
Meter repeatability	< 10%	3.6%
%Capture of phase energy	≥ 90%	phase 3
Predicted K* vs measured K*	within ±1	phase 5

The last row is the falsifiable test. We build an analytic energy model from the component costs and the acceptance curve, predict K* **without** running the sweep, then compare. If it predicts, the attribution explains the curve. If it does not, the decomposition is incomplete and we report that.

THREATS, AND WHAT WE DO ABOUT THEM

Threat	Mitigation
Clocks boost and throttle, and cannot always be locked	fixed warm-up, repeated trials, median reported; variance measured, not assumed
Idle draw drifts between runs and cards	floor measured per process per run; both accountings reported side by side
Results are card-specific — now confirmed	measured, not assumed: the same model on two cards inverts the ordering, so every K* we report names its device and power cap
One dataset, one task family	stated as a limit; CNN-DM and HumanEval already wired into the loader

Differences reported as paired per-prompt deltas with 95% confidence intervals, never as overlapping point estimates

Where the project stands

Phase	Owner	State
1 · Setup and baseline generation	All	Done
2 · Draft–verify–rollback engine, correctness	Anupam	Done · losslessness proven
3 · K sweep: acceptance, latency, energy	Kanishk	Done · 3 configurations, all complete
4 · CLEAR activation store and amplification	Ankit	Next
5 · Component attribution and the K* prediction test	All	K* found; attribution pending

IMMEDIATE NEXT STEPS

- Build the activation store and report a first %Capture on one component
- Attribute the power-cap effect to components: which ones drop power as K grows?
- Extend the NF4 sweep past K=8 so its optimum falls inside the measured range

MAIN METHODOLOGICAL RISK

%Capture may fall below 90% on a two-model pipeline, where cache rollback and control flow sit between components. **Mitigation:** coarsen the component boundaries until the sum closes, and report the granularity actually achieved rather than the one we wanted.

IF ATTRIBUTION FAILS

The project still delivers a phase-granular energy-vs-K characterisation that is measured, paired and lossless-gated. Component attribution upgrades that from a measurement to an explanation; it is not what the result rests on.

Code, runbook and all results: github.com/a-n-u-p-a-m/PAA_PROJECT